

AI MODELS COMPARATOR

Software Design Description (SDD)

Software Engineering – A4 – 2026-2027

Vivien Gruel
Noé Wales
Valentin Gempp
Maxence Tessier

Contents

1	Introduction	2
2	System Overview	2
3	Architectural Views	2
3.1	Context View	2
3.2	Component View	3
3.3	Deployment View	4
4	Data Design	5
5	Interface Design	6
5.1	REST API	6
5.2	User Interface	6
6	Detailed Design	7
6.1	Component Responsibilities	7
6.2	Sequence – Benchmark Import	7
6.3	State Machine – User Account	8
6.4	Error Handling and Recovery	9
7	Security and Compliance	9
8	Performance, Scalability, Availability	10
9	Observability	10
10	Internationalization and Accessibility	10
11	Design Rationale	10
12	Assumptions and Constraints	11
13	Open Issues and Future Work	11

1 Introduction

Purpose. This document explains **how** the AI Models Comparator will be built. It describes the architecture, the data, the interfaces and the main technical choices, so that the developers can implement the system and the testers can check it.

Scope. The AI Models Comparator is a responsive web application. It shows AI models grouped by lab, a benchmark ranking, a community ranking based on up and down votes, a comment feed for each model, AI news, and an administration page.

Audience. Developers, testers and the TD instructor.

References. IEEE 1016-2009 (SDD), Business Requirements Document (BRD) and Software Requirements Specification (SRS) of the AI Models Comparator, SHMS SDD template given in the course.

2 System Overview

The AI Models Comparator is a classic three-tier web application:

- a **frontend** in React, which runs in the browser of the user;
- a **backend** in Node.js with Express, which contains the business logic and exposes a REST API;
- a **PostgreSQL database**, which stores all the data.

The backend also runs scheduled jobs that import benchmark scores and AI news from external websites. The users never call these websites directly: they only read the data saved in our database. This way, the website still works when an external website is down.

3 Architectural Views

3.1 Context View

The system is between the human actors (visitors, registered users, administrators) and two kinds of external systems (benchmark source and news feed).

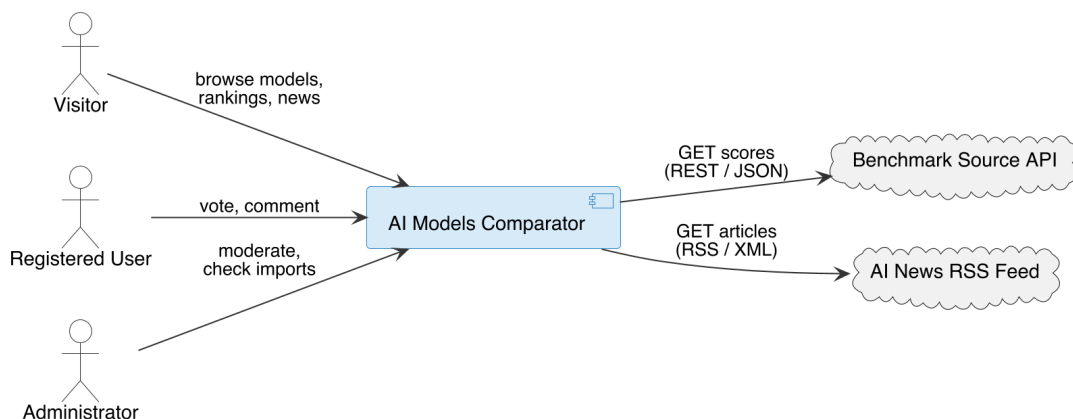


Figure 1: Context diagram

Main interactions:

- **Visitors ↔ system:** browse models, rankings and news, sign up, log in.

- **Registered users** ↔ **system**: vote, comment, delete their comments and their account.
- **Administrators** ↔ **system**: delete comments, block, unblock and delete users, check the imports.
- **System** → **benchmark source**: get the benchmark scores (REST, JSON).
- **System** → **news feed**: get the latest news (RSS, XML).

3.2 Component View

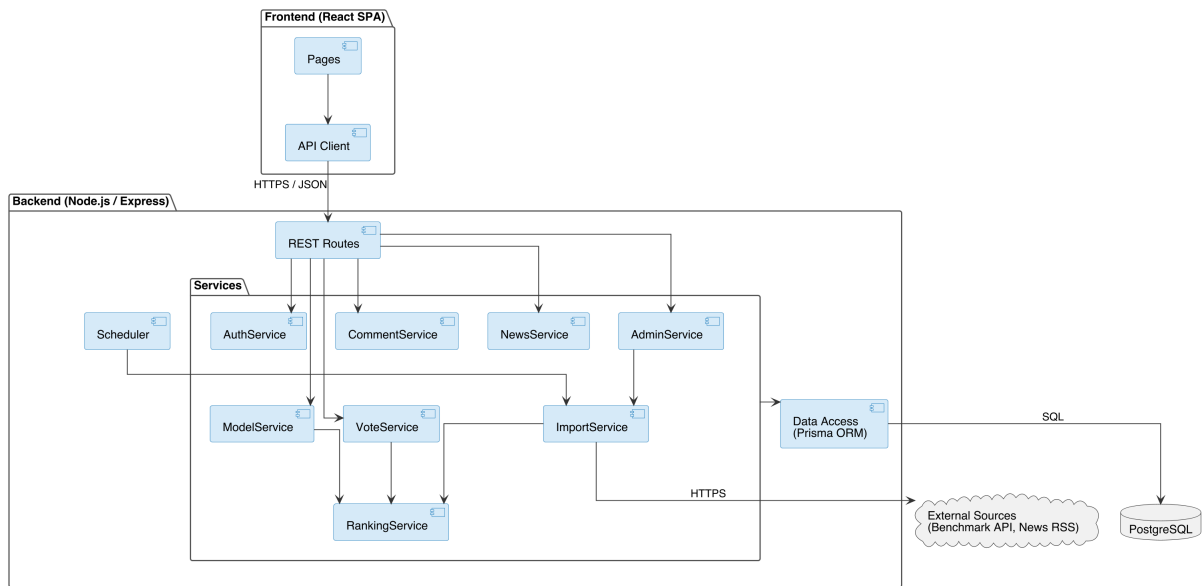


Figure 2: Component diagram

Components:

- **Pages** (frontend): one React component for each page of the SRS.
- **API Client** (frontend): sends the HTTP requests to the backend.
- **REST Routes**: receive the requests, check the session and the role, validate the inputs, and call the right service.
- **AuthService**: sign up, log in, log out, session management.
- **ModelService**: labs, models and model details.
- **VoteService**: add, change and remove votes.
- **CommentService**: add, list and delete comments.
- **RankingService**: computes the community and benchmark scores and the ranks.
- **NewsService**: lists the news.
- **AdminService**: moderation of comments and users, import status.
- **ImportService**: downloads, checks and saves the data from the external sources.
- **Scheduler**: starts the imports at fixed times (node-cron).

- **Data Access:** Prisma ORM, the only component that talks to the database.

Main dependencies: all services use the Data Access layer. VoteService, ModelService and ImportService use RankingService. The Scheduler and AdminService start the ImportService.

3.3 Deployment View

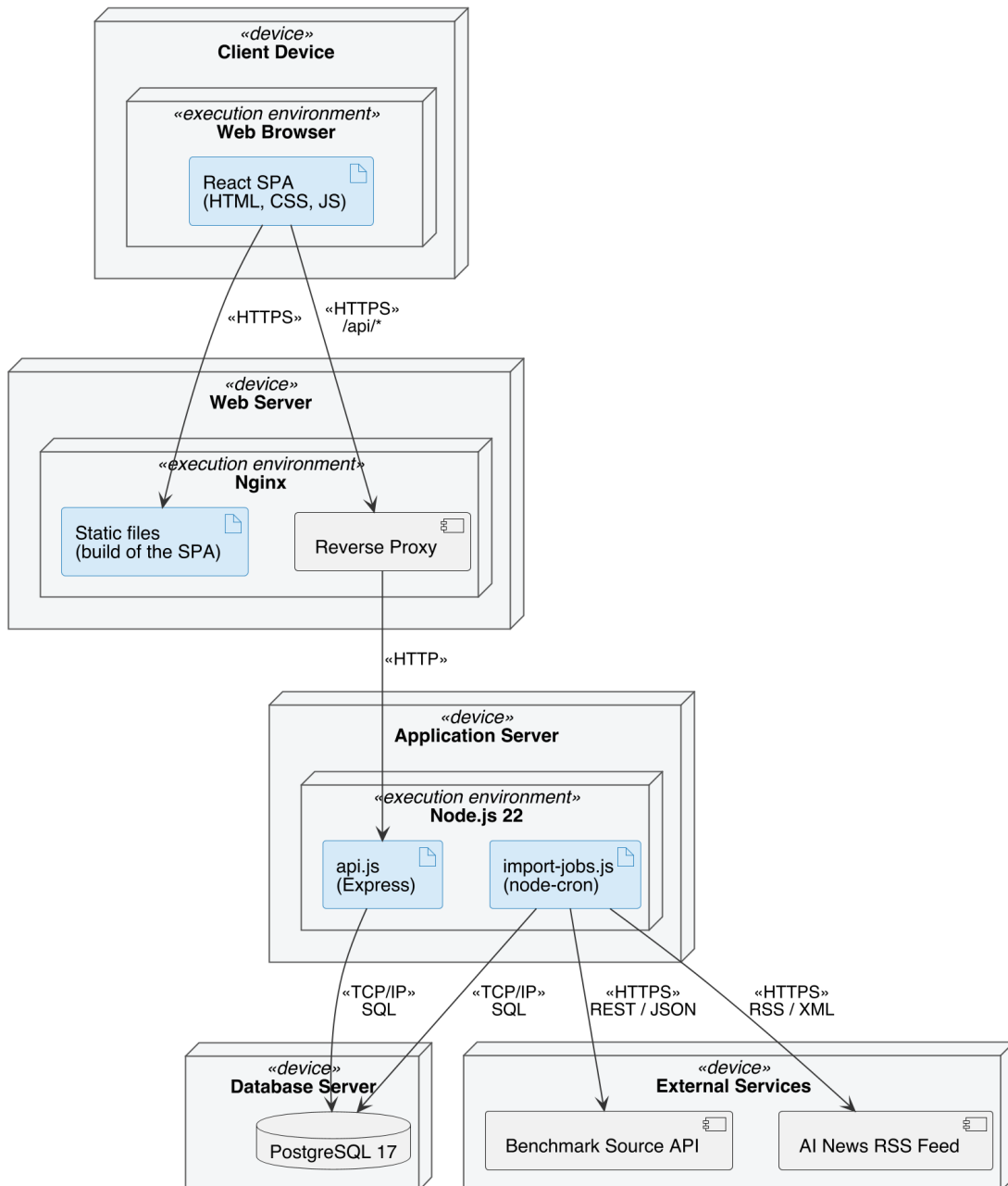


Figure 3: Deployment diagram

- **Client device:** the React SPA runs in the web browser of the user (computer or phone).
- **Web server:** Nginx serves the static files of the SPA and forwards the `/api/*` requests to the application server. HTTPS is enabled with a Let's Encrypt certificate.
- **Application server:** Node.js 22 runs the Express API and the import jobs.

- **Database server:** PostgreSQL 17, saved every day.
- **External services:** benchmark source API and AI news RSS feed, called only by the import jobs.

For the course, the web server, the application server and the database can run on the same machine with Docker Compose (one container for each).

4 Data Design

Main tables:

- **users**(id, email, username, password_hash, role, status, created_at)
- **labs**(id, name, logo_url)
- **models**(id, lab_id, name, slug, release_date)
- **votes**(user_id, model_id, type, updated_at)
- **comments**(id, user_id, model_id, content, created_at)
- **benchmarks**(id, name, description)
- **scores**(model_id, benchmark_id, source_id, value, fetched_at)
- **sources**(id, name, url, type, last_success_at, last_error)
- **news**(id, source_id, title, url, published_at)

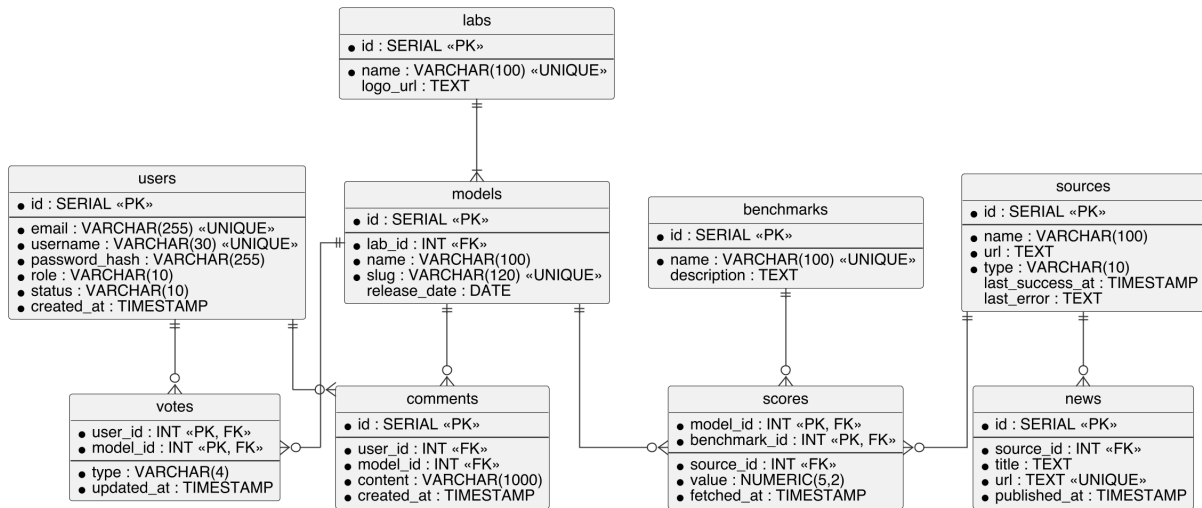


Figure 4: Entity-relationship diagram

Integrity rules:

- The primary key of **votes** is (user_id, model_id): a user can only have one vote per model.
- The primary key of **scores** is (model_id, benchmark_id): a model has only one score per benchmark.

- `votes.type` is UP or DOWN; `users.role` is USER or ADMIN; `users.status` is ACTIVE or BLOCKED (CHECK constraints).
- `scores.value` is between 0 and 100.
- `email`, `username` and `models.slug` are unique.
- When a user is deleted, their votes and comments are deleted too (ON DELETE CASCADE).
- A lab or a model cannot be deleted if it still has models, votes, comments or scores (ON DELETE RESTRICT).

5 Interface Design

5.1 REST API

All routes start with `/api`, use JSON, and return standard HTTP codes.

Method	Route	Description	Access
POST	<code>/auth/register</code>	Create an account	Public
POST	<code>/auth/login</code>	Log in, set the session cookie	Public
POST	<code>/auth/logout</code>	Log out	User
GET	<code>/auth/me</code>	Current user	User
DELETE	<code>/users/me</code>	Delete my account	User
GET	<code>/labs</code>	Labs with their models	Public
GET	<code>/models/{slug}</code>	Model details, scores, ranks, votes	Public
GET	<code>/rankings?type=community benchmark</code>	Ranking	Public
POST	<code>/models/{slug}/vote</code>	Add, change or remove my vote	User
GET	<code>/models/{slug}/comments</code>	Comments, newest first	Public
POST	<code>/models/{slug}/comments</code>	Add a comment	User
DELETE	<code>/comments/{id}</code>	Delete a comment (own, or any for admin)	User
GET	<code>/news</code>	AI news, newest first	Public
GET	<code>/admin/users</code>	List of users	Admin
PATCH	<code>/admin/users/{id}</code>	Block or unblock a user	Admin
DELETE	<code>/admin/users/{id}</code>	Delete a user	Admin
GET	<code>/admin/comments</code>	Recent comments	Admin
GET	<code>/admin/sources</code>	Import status of each source	Admin

Example: POST `/api/models/gpt-5/vote` with `{"type": "UP"}` returns 200 with `{"score": 82.4, "voteCount": 1204, "myVote": "UP"}`.

5.2 User Interface

- **Public pages:** home, models, model page, ranking, news, login.
- **User pages:** account page; vote and comment forms on the model page.
- **Admin page:** users, recent comments, import status.

The wireframes are in the SRS (Section 6.1). The interface is built with Tailwind CSS and is responsive (mobile first).

6 Detailed Design

6.1 Component Responsibilities

- **VoteService**
 - Finds the existing vote of the user for the model.
 - No vote: creates it. Same type: deletes it. Other type: updates it.
 - Asks RankingService for the new community score.
- **RankingService**
 - Community score = $\text{up} / (\text{up} + \text{down}) \times 100$, only if the model has at least 10 votes.
 - Benchmark score = average of the scores, only if the model has at least 3 scores.
 - Sorts the models by score; in case of equality, the model with more votes (or scores) comes first.
- **ImportService**
 - One importer for each source, with the same interface: `fetch()`, `validate()`, `save()`.
 - Validates the data with Zod before saving it.
 - Saves everything in one transaction: if something fails, nothing is changed.
- **AuthService**
 - Hashes the passwords with bcrypt.
 - Creates a JWT stored in an HttpOnly cookie (valid 7 days).
 - Refuses the login if the account is **BLOCKED**.

6.2 Sequence – Benchmark Import

1. Every day at 3:00, the Scheduler starts the benchmark import.
2. For each benchmark source, ImportService calls the external API.
3. If the data is valid, ImportService saves all the scores in one transaction. Unknown models are added to the catalogue.
4. If there is an error, the old scores are kept and the error is saved in the `sources` table.
5. At the end, the benchmark ranking is computed again.

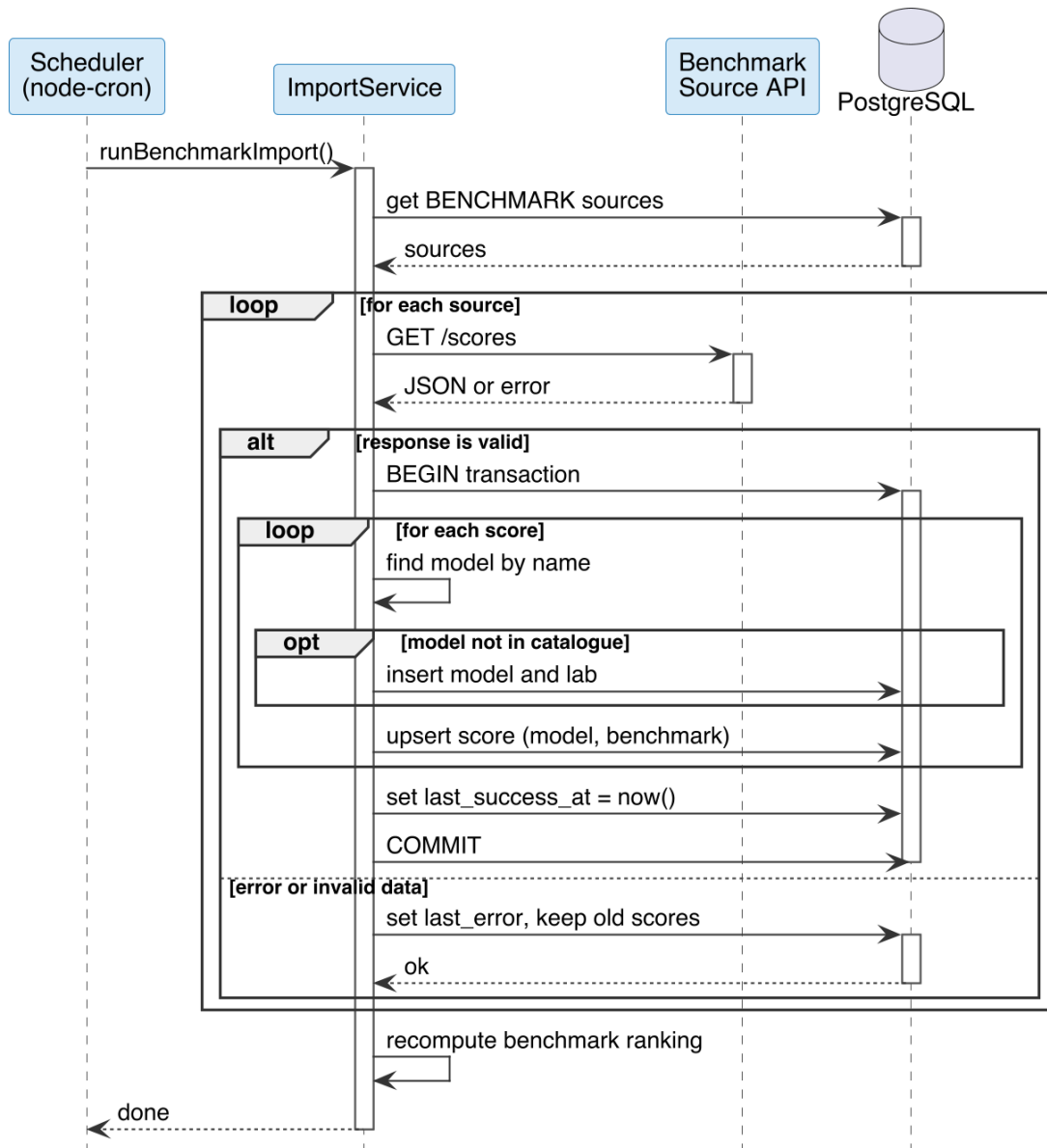


Figure 5: Sequence diagram – Benchmark import

6.3 State Machine – User Account

A user account is **ACTIVE** after sign up. Inside this state, the user is logged in or logged out. An administrator can block the account: all sessions end and the user cannot log in anymore. The account can be unblocked, or deleted.

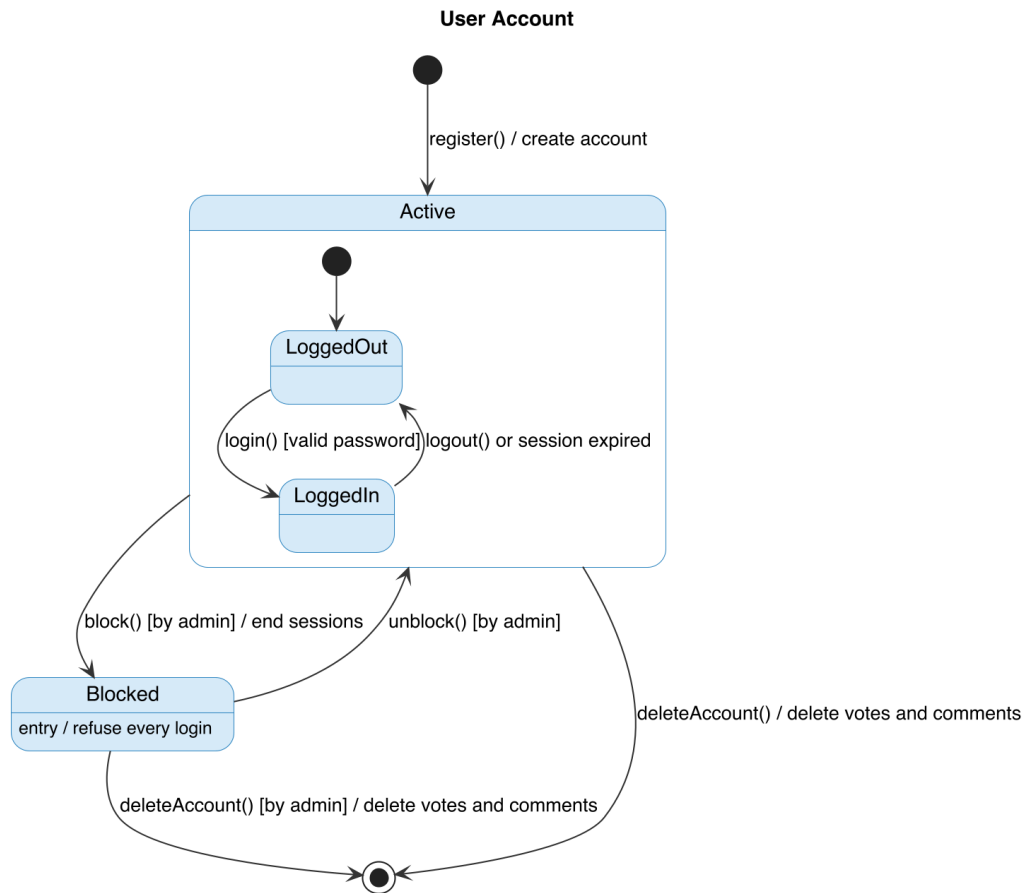


Figure 6: State machine diagram – User account

6.4 Error Handling and Recovery

- **400** for invalid data (for example an empty comment), with a clear message.
- **401** if the user is not logged in, **403** if the user does not have the right role or is blocked.
- **404** if the model, the comment or the user does not exist.
- **409** if the email or the username is already used.
- **500** for unexpected errors: a global Express error handler returns a generic message and logs the details.
- Import errors never stop the website: the last valid data is kept.

7 Security and Compliance

- Passwords hashed with bcrypt; JWT in an HttpOnly, Secure, SameSite cookie.
- Role-based access control (visitor, user, admin) checked on every route by an Express middleware.
- HTTPS everywhere; security headers with Helmet; CORS limited to our frontend.
- Input validation with Zod; SQL injection avoided thanks to Prisma (parameterised queries).

- Comments are displayed as plain text by React, which protects against XSS.
- Rate limiting on login and sign up to limit brute-force attacks and fake accounts.
- **GDPR:** only email, username and hashed password are stored; consent at sign up; account deletion removes all personal data, votes and comments.

8 Performance, Scalability, Availability

- Rankings are computed after each import or vote and not at each page view.
- Database indexes on `models.slug`, `votes.model_id`, `comments.model_id` and `scores.model_id`.
- Comments are loaded by pages of 20.
- The API is stateless (JWT), so more Node.js instances can be added behind Nginx if needed.
- Target: less than 2 seconds for each page with 100 users at the same time.

9 Observability

- JSON logs for each request (method, route, status, duration) with the library Pino.
- Each import saves its result in the `sources` table (last success, last error), visible on the admin page.
- A GET `/api/health` route checks that the API and the database work.

10 Internationalization and Accessibility

- The interface is in English at the start. All the texts are in one file, so another language can be added later.
- Dates are displayed in the format of the browser.
- Accessibility: labels on all form fields, keyboard navigation, good color contrast, and information never given only by color (for example, the selected vote arrow also changes its shape).

11 Design Rationale

- **React + Node.js/Express:** required by the course, and the same language (JavaScript/TypeScript) on the frontend and the backend.
- **PostgreSQL:** relational data with strong constraints (one vote per user and model, cascade delete), which is important for our rules.
- **Prisma:** simple ORM, safe queries and database migrations.
- **Imports saved in our database:** the website is faster and still works when an external source is down.
- **One service per feature:** the team can work in parallel, and a change in one feature does not break the others.

12 Assumptions and Constraints

- Web application only; no native mobile app.
- The external sources are free and allow programmatic access.
- Team of four students, with the time of the course sprints.
- Hosting on one small server (or on the computers of the team for the demo).

13 Open Issues and Future Work

- Final choice of the benchmark source and of the news feed.
- Automatic matching of model names between sources (today, a simple name comparison).
- Detection of fake accounts and vote manipulation.
- Replies to comments and reports of bad comments by the users.
- History of the rankings over time.